

GeoPKI

I. MODEL

The earth ellipsoid model of the existing world geodetic system 84 (WGS84) [1] is used. WGS84 defines the *semi-major axis* (half of the largest diameter of an ellipsoid, at the equator) a to be 6'378'137.0 meters [1]. The *semi-minor axis* (the polar radius) b is defined as 6'356'752.3142 meters [1]. Standard polar coordinates (longitude, latitude) are used for referencing points on the planet's two-dimensional surface and a third coordinate, altitude, extends this to three dimensions. Longitude, latitude and altitude together refer to a unique point in three-dimensions.

Each of the three coordinates is discretized in a way that preserves meter-precision in the worst case. With the semi-major axis a , the maximum circumference of the ellipsoid equals $= 2a\pi$ meters. Using base two, $B_x := \lceil \log_2(2a\pi) \rceil = 26$ bits for the longitude thus suffice to represent each possible meter value in the worst case. Let $C_x := 2^{B_x} - 1$ denote the maximum discretized value for the longitude. For the latitude the circumference is $2b\pi$ meters but we only need to be able to represent half of it, so $b\pi$. Thus $\lceil \log_2(C_y) \rceil = 25 =: B_y$ bits suffice. Analogously to C_x , let $C_y := 2^{B_y} - 1$ the maximum discretized value for the latitude. For altitude there is no natural bound so one has to be fixed. At the time of writing, the highest point on earth is below 10'000 meters above sea level [2] and the lowest layer of earth's atmosphere, the troposphere, extends to about 18 km [3]. The Mariana Trench at about 10'000 m below sea level [4] is likely a bound not exceeded in the near future. The necessity of allowing volumes below sea level arises from countries such as the Netherlands where parts of the surface's altitude is below sea level [5]. Moreover in the future, below sea level research stations might be built. Thus the range starting at a depth of $D := -10'000$ m up to an altitude of 20'000 m above sea level should cover any current and near-future needs. Covering this range requires $B_z := \lceil \log_2(H - D) \rceil = 15$ bits. In fact using B_z bits starting at D will cover the range from D to $D + (2^{B_z} - 1) = 22'767 := H$. Complications might arise from distance calculations when using sea level as a reference as the sea level is not the same in all places and will increase in the future. Thus instead of using sea level as a reference, the geodetic altitude defined by WGS84 [1] is used. Given the geoid's surface in WGS84 is about at the mean sea level (MSL), the range $[D, H]$ can be left as is. While it does not exactly cover the range described range with respect to local sea level, it approximates it.

II. DATA STRUCTURE

Inspired by F-PKI [6], a sparse merkle hash tree (SMT) is used. Each leaf represents a full discretized coordinate using $B := B_x + B_y + B_z = 66$ bits resulting in a total of 2^B leaves. To have spatially close points somewhat close together in the SMT, the space-filling Z -order curve is used to sort the

three dimensional locations in one dimension [7]. An example with $B_x = B_y = B_z = 2$ is shown in Figure 1a. GeoPKI uses a slightly modified 3D Z -order curve: Rather than using the full 3D Z -order curve, a 2D Z -order curve over the longitude and latitude is used and points with the same longitude and latitude but different altitude are put in sequence. While this leads to spatially close points being further apart in the 1-dimensional sorting, this modification allows for shorter proofs under some assumptions (will be discussed in Section IV) and even results in a nicer binary representation. Figure 1b contrasts this modification with the standard 3D Z -order curve shown in Figure 1a.

A. Leaf Binary Format

Each leaf of the SMT corresponds to a point and the tree leaves are sorted lexicographically by their binary representation in ascending order. A point tuple $p := (x, y, z)$ at longitude x , latitude y and altitude z with

$$(x, y, z) \in [-180, 180] \times [-90, 90] \times [D, H]$$

is first discretized to integer coordinates as follows:

$$\begin{aligned} p' &:= (x_d, y_d, z_d) \\ &= \left(\left\lfloor \frac{x + 180}{360} \cdot C_x \right\rfloor, \left\lfloor \frac{y + 90}{180} \cdot C_y \right\rfloor, \lfloor z - D \rfloor \right) \end{aligned} \quad (1)$$

After this transformation the following holds:

$$p' \in [0, 2^{B_x}) \times [0, 2^{B_y}) \times [0, 2^{B_z})$$

This formula holds since the first term in the longitude and latitude tuples can be at most 1 and since C_x and C_y are the maximum values that can be represented using B_x and B_y bits, respectively. Moreover $H - D = 2^{B_z} - 1$ holds meaning the altitude can be represented using B_z bits. The three resulting integer numbers can now be represented using B_x , B_y and B_z bits, respectively. Let $p'' := (x_b, y_b, z_b)$ be their big-endian integer binary encoding with

$$p'' \in \{0, 1\}^{B_x} \times \{0, 1\}^{B_y} \times \{0, 1\}^{B_z}$$

For $c \in \{x, y, z\}$ and for any integer $i \in \{0, \dots, B_c - 1\}$, let $c_{b,i}$ denote the i -th bit of c_b . Moreover let $\parallel$ denote the concatenation operator. The binary representation p_b of the point p is then defined as

$$p_b = \left(\bigparallel_{i=0}^{B_y-1} x_{b,i} \parallel y_{b,i} \right) \parallel x_{b,B_x-1} \parallel \left(\bigparallel_{i=0}^{B_z-1} z_{b,i} \right)$$

In words, the binary representation of p is the interleaving of the binary representation for the longitude and latitude concatenated to the binary representation of the altitude.

Figure 2a shows an example of this binary encoding.

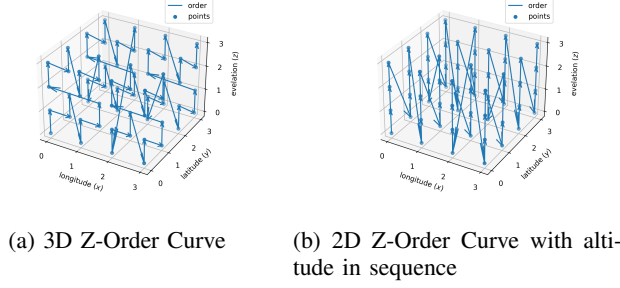


Fig. 1: Difference between a standard 3D Z-order curve and the one used by GeoPKI when using two bits for all coordinates.

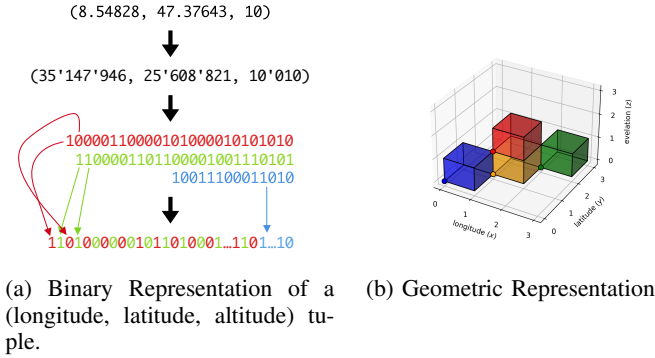


Fig. 2: The binary and geometric representation of a leaf

B. Leaf Geometric Representation

Due to the floor functions in Equation (1), the point p'' represents the volume where p'' is in the corner with the smallest coordinates in all three dimensions. By replacing the floor functions by mathematically rounding or the ceil function the volumetric representation changes. In case of the floor functions shown in Equation (1), the point corresponding to p'' is

$$\left(x_d \cdot \frac{360}{C_x} - 180, x_d \cdot \frac{180}{C_y} - 90, z_d + D \right) \quad (2)$$

and the volume represented by point p'' is given by

$$\left[x_d \cdot \frac{360}{C_x} - 180, (x_d + 1) \cdot \frac{360}{C_x} - 180 \right) \times \left[y_d \cdot \frac{180}{C_y} - 90, (y_d + 1) \cdot \frac{180}{C_y} - 90 \right) \times [z_d + D, z_d + D + 1) \quad (3)$$

Figure 2b shows the points and the corresponding volumes in the same color.

C. Intermediate Nodes

Intermediate nodes in the SMT also correspond to a point and a volume following the same pattern as for the leaf nodes. The bit string corresponding to an intermediate node is simply the longest common prefix of it's two children and covers the

whole volume of its two children. The left child is the node with the binary representation equal to the intermediate node one's concatenated with a zero and the right child concatenated with a one. The formula for computing the volume becomes more complex for intermediate nodes as it requires a case distinction: Let s be the bit string of an intermediate node.

Let l be the length of s . Let s_x, s_y, s_z be the bit prefixes encoded in s corresponding to the longitude, latitude and altitude, respectively. Moreover let l_x, l_y and l_z be the lengths of s_x, s_y and s_z , respectively, and let $s_{c,i}$ for $i \in \{0, \dots, l_c - 1\}$ and $c \in \{x, y, z\}$ correspond to the i -th bit of s_c . It holds that $l = l_x + l_y + l_z$ and it is possible that l_x is by exactly one larger than l_y , otherwise $l_x = l_y$. This directly follows from the interleaved binary encoding described in Section II-A. Let x'_d, y'_d, z'_d be the most-precise integers that can be re-constructed with the given bits:

$$x'_d := \sum_{i=0}^{l_x-1} 2^{B_{x,y}-1-i} \cdot s_{x,i} \quad (4)$$

$$y'_d := \sum_{i=0}^{l_y-1} 2^{B_{x,y}-1-i} \cdot s_{y,i} \quad (5)$$

$$z'_d := \sum_{i=0}^{l_z-1} 2^{B_z-1-i} \cdot s_{z,i} \quad (6)$$

Then the corresponding point is given by

$$\left(x'_d \cdot \frac{360}{C_x} - 180, y'_d \cdot \frac{180}{C_y} - 90, D + z'_d \right) \quad (7)$$

and the covered volume by

$$\left[x'_d \cdot \frac{360}{C_x} - 180, (x'_d + 1) \cdot \frac{360}{C_x} - 180 \right) \times \left[y'_d \cdot \frac{180}{C_y} - 90, (y'_d + 1) \cdot \frac{180}{C_y} - 90 \right) \times [D + z'_d, D + (z'_d + 1)) \quad (8)$$

This is the more general form of Equation (3) and for $l_x = l_y = B_{x,y}$ and $l_z = B_z$ the two are equivalent.

It is worth nothing is that in GeoPKI, intermediate nodes also store a set of certificates. When using a tree representing the geography (which can reduce proof sizes as described in Section V), this is almost a requirement. Without this optimization, almost no leaf would be empty as almost all pieces of land on earth are claimed by at least one country. This would require enormous storage costs and result in less efficient queries.

D. Sparse Merkle Hash Tree (SMT)

The just described tree is combined with a SMT. As in F-PKI [6], each leaf stores a set of certificates. Let $C_0, C_1, \dots, C_{n-1}$ be the sequence of certificates located in a leaf, sorted lexicographically by their SHA-256 hashes in ascending order. The hashes are computed on the PEM encoding of the X.509 certificates. *cyrill: sort by sha256 hash and then by binary representation? (should never occur)*

though as then we would have bigger problems due to a sha-256 collision in a certificate...) *nico*: I hope the sentence clearer now? I just wanted to clearly specify what the hash is computed on. It could be the decoded binary value, the base64 encoding, the PEM encoding, the DER encoding, ... The leaf's hash value is then defined as

$$H(0||H(C_0)||H(C_1)||\dots||H(C_n)) \quad (9)$$

where H is SHA-256. The hash value of an empty / non-existent leaf in the SMT is $H(0)$. *cyrrill*: are empty nodes constructed iteratively starting from the leaf, i.e., $H(0)$ at depth $B_x + B_y + B_z$? Or does an intermediate node with value $H(0)$ at any depth represent an empty subtree? *nico*: I think F-PKI constructs them iteratively, right? I don't think it matters too much or did you discuss any advantages / disadvantages when deciding that? Using the same value at any depth can save some computations but I have not properly thought about the implications this would have.

An intermediate node in the tree has two children (two intermediate nodes or two leaves) and in GeoPKI also stores a set of certificates. As before, let $C_0, C_1, \dots, C_{n-1}$ be the sequence of certificates located in an intermediate node, sorted lexicographically by the SHA-256 hashes of the PEM encoding in ascending order. Hash h_3 over the list of certificates is defined as

$$h_3 := H(H(C_0)||H(C_1)||\dots||H(C_n)) \quad (10)$$

Then let h_1 and h_2 be the hash values of the left and right child of the intermediate node, respectively. The hash value of the intermediate node is then defined as

$$\begin{cases} H(1||h_1||h_2) & \text{if } n = 0 \\ H(1||h_1||h_2||h_3) & \text{if } n > 0 \end{cases} \quad (11)$$

The constants 0 and 1 are prefixed in the hash computation to prevent collisions where a given hash $H(a||b)$ for some a and b could simultaneously refer to a leaf with content $a||b$ or a intermediate node with children hash values a and b .

E. Querying the SMT

Since many sizes of volumes can be represented by the binary format described in Sections II-A and II-C, the only type of query a GeoPKI map server supports is such a bit string. Such a bit string has a unique corresponding node in the tree (that might not exist as the tree is sparse). The result then is the set of all certificates stored at intermediate nodes above the specified node (as these represent larger areas covering the queries point) and the set of all certificates stored in the subtree rooted at the specified node as these are within the queried area. The former set corresponds to the set of nodes whose corresponding binary representation are a prefix for the given query. The latter set corresponds to the set of nodes whose corresponding binary representation the given query is a prefix for.

A trivial way to obtain this set of certificates is to walk the tree from the root and collect the certificates stored at the

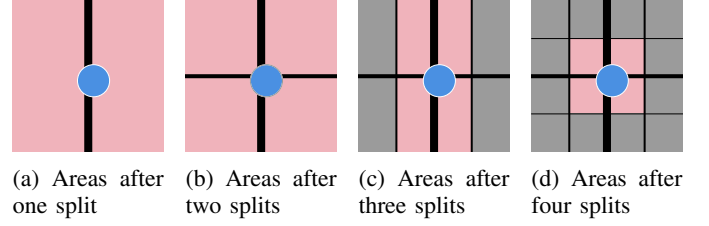


Fig. 3: The two-dimensional surface (longitude, latitude) after the first, second, third and fourth split. The blue circle shows a possible area to be claimed by a certificate or the desired query area of a client. The area colored in red is the minimum area that has to be retrieved using the given number of splits.

intermediate nodes along the way. If the next bit is a zero, go left, if it is one, go right. Once the node with a bit string corresponding to the query is found, the full subtree rooted at this node is walked recursively until an empty node or a leaf is found and all found certificates are returned.

F. Arbitrary Volumes and Certificate Placement

The discretization used by GeoPKI does not directly fit all requirements. On one hand, claimed volumes will likely never fit exactly one of the voxels. Moreover, clients likely want to query volumes not aligning with the GeoPKI voxels.

In both cases this volume V has to be approximated using the GeoPKI voxels. A trivial approximation the smallest volume encompassing all of V . Unfortunately this results in returning the whole tree if V lies at the intersection of one of the first few division lines. Figure 3 depicts this situation with the blue circle corresponding to V and the red area the minimum area that has to be retrieved at the different levels in the tree. Figure 3a shows the line of the first split. Given the blue circle intersects both area, the above trivial approximation would result in the whole tree being returned for a comparable small area that is just placed unfortunately.

Thus a better approximation is required. The main problem with the trivial approximation is the discrepancy of the area queried and the returned area: If the client queries for a large area, it is fine to return a lot of data. But in case of a query for a small area, the size of the returned data should be in relation to the queried area. Given the split lines do not move when going to a deeper level in the tree, it is always necessary to return multiple voxels. The number of required voxels is monotonically increasing when going deeper down in the tree but the area covered by them monotonically decreases as the voxel approximation for V becomes more accurate. With decreasing area, the data that has to be returned also monotonically decreases as the subtrees contain at most as much data as any of their super-trees.

1) *Claimed Volumes to Voxels*: In case of the placement of certificates in the tree, it is infeasible to place certificates claiming a volume V in all leaves covered by V due to the loss in sparsity and the resulting explosion in storage requirements. As noted before, most places on earth are claimed by at least one country and thus the tree would not be sparse at all.

A plausible heuristic balancing the returned data size with the number of subtrees loss in sparsity is to use a subtree level where the volume of V is greater than or equal to a given fraction $f \in [0, 1)$ of the voxel volumes at this level. For instance $f = \frac{1}{4}$ would result in Figure 3c and $f = \frac{1}{2}$ in Figure 3d (assuming all shown areas cover the full altitude spectrum). The effectiveness of this heuristic and a good value for f is to be determined using empirical experiments.

2) *Query to Voxels*: For the transformation of queries to voxels to be queried the case turns out to be easier. In terms of response size it is always best to retrieve all individual leafs as directly querying a area higher up in the tree will always be a superset of that data.

G. Proofs of Presence and Absence

Given a query for a volume V , the map server should prove that the returned data is in fact in the SMT and that no certificate was omitted. As described in Section II-E, the server directly only supports queries for bit strings that have a unique corresponding node v in the SMT. Let C with $C \in C$ be the set of all certificates stored at v . The server has to send the set $\{H(x) \mid \forall x \in C\}$ to the client as it is required to compute the node's hash (see Equations 9 and 10). As described in Section II-E, if v is an intermediate node, the query result will include the whole subtree rooted at v . Hence a client has all information to compute the hashes of v 's children by themselves. These are required to compute the hash of an intermediate node (see Equation 11).

What is left to complete the proof is the set of complementary hashes along the way to the root and the root hash itself as in a standard SMT. Since in GeoPKI the intermediate nodes along the way to the root also have associated certificates, the set of their hashes is also required but as specified in Section II-E, this information is returned to the client anyway and can thus be computed locally.

In a sparse SMT, proving the absence of any certificate in a subtree rooted at v is the same as proving the presence of an empty node. Thus when a client receives a shorted path up to the root than expected, it already has all information required to proof the node's absence in the tree.

Given the binary SMT has 2^B leaves (see Section II), it's depth is B . A proof of presence for a node v at depth $d \in \{0, \dots, B-1\}$ (zero is the root itself) then consists of the d complementary node's hash along the path to the root. The size of these hashes is 256 bits or 32 bytes due to the use of SHA-256. Thus in the worst case a proof for a single leaf consists $(B-1) \cdot 32 \text{ B} = 66 \cdot 32 \text{ B} = 2112 \text{ B} \approx 2 \text{ kB}$ and is thus in the same order as an average X.509 certificate. The size of such a proof is smaller the sparser the tree is and is always smaller if an intermediate node, i.e., not a leaf, is queried.

H. Proof of Consistency

As in F-PKI [6] using a consistency tree over the root hashes.

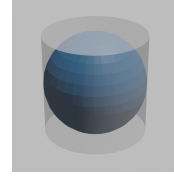


Fig. 4: Approximating a sphere using a cylinder.

III. SERVER IMPLEMENTATION

Due to the size of data it seems reasonable to use a data management engine on the server's side, storing the whole data structure in memory is infeasible at the time of writing.

A. Querying for a Sphere Efficiently

One possibility to query a sphere is to walk the tree and intersect it with the volumes on each level of the tree. This process is repeated recursively until the reached level is a (sparse) "leaf". When walking the tree like this, all information that has to be returned (including the data for the proof) is obtained in one go. The database management systems MySQL and PostgreSQL could be extended with a custom index performing the just described operations but they already have built-in support for spatial indexes.

Both have support for two dimensional geodetic coordinates, PostgreSQL additionally supports three dimensional cartesian coordinates but none of them supports two dimensional geodetic coordinates plus a third cartesian altitude dimension. Transforming points to earth-centered, earth-fixed (ECEF) cartesian coordinates is straight-forward but the same does not apply to volumes since the coordinate system's grid lines are different. In the geodetic coordinate systems grid lines are curved while in a cartesian coordinate system they are straight lines. Hence the shape of volumes defined by their vertices is wildly different and would need to be over-approximated when wanting to use the three dimensional spatial index. While possible, such over-approximations do not seem to be trivial to compute.

Alternatively it is possible to directly make use of the two-dimensional spatial index with support for geodetic coordinates. In this approach the altitude is initially completely ignored and the voxel's two dimensional projection to the earth's surface is intersected with a circle with the sphere's radius whose center is the sphere's center also projected to earth's surface. After querying the spatial index for this, the returned data can be filtered based on the altitude. Note that this type of query no longer searches for volumes intersecting a sphere but rather over-approximates the sphere as an extruded circle, i.e. a cylinder (see Figure 4). Due to the projection to the earth's surface, the radius of the cylinder has to be adjusted. When projecting two-dimensional shapes defined by vertices from higher altitudes to the earth's surface (by keeping the longitude and latitude but omitting the altitude), their area shrinks. Geodetic coordinates simply express the location on the two-dimensional ellipsoid surface where with increasing altitude the corresponding ellipsoid and thus surface area grow.

Analogously when projecting shapes at negative altitudes, their area grows. This needs to be taken into account when performing the intersection with a circle on earth's surface. Assuming the original sphere that is queried for is below WGS84's ellipsoid surface and has a radius r , the radius used for computing the intersection on earth's surface has to be greater than r since the distance between the sphere's center and all other points on that altitude has increased. Analogously, the radius can be decreased if the sphere's altitude is above the ellipsoid's surface but in this case it is not necessarily required as over-approximating the result is fine when only considering correctness. Since it is the distances and not areas or volumes that are of interest, the circumferences of ellipses with the different radii should be compared. Given there is no easy formula for the circumference of an ellipse, the difference is over-approximated by considering spheres instead. The circumference of an ellipsoid is greater than the circle with a radius corresponding to the length of the ellipse's semi-minor axis (b for WGS84). Similarly, an ellipsoid's circumference is smaller than the circle with a radius corresponding to the length of the ellipse's semi-major axis (a for WGS84). Thus the fraction by which distances grow from altitude h with $h < 0$ to the earth's surface can be over-approximated as

$$\begin{aligned} & \frac{\text{circumference of WGS84 ellipse at } z = 0}{\text{circumference of WGS84 ellipse at } z = h} \\ & \leq \frac{\text{circumference of circle with radius } a}{\text{circumference of circle with radius } (b + h)} \\ & = \frac{2a\pi}{2(b+h)\pi} = \frac{a}{b+h} \end{aligned} \quad (12)$$

For $h = D$ this results in ≤ 1.005 .

Analogously the fraction by which distances shrink from altitude h with $h > 0$ to the earth's surface can be over-approximated as

$$\begin{aligned} & \frac{\text{circumference of WGS84 ellipse at } z = h}{\text{circumference of WGS84 ellipse at } z = 0} \\ & \leq \frac{\text{circumference of circle with radius } (a + h)}{\text{circumference of circle with radius } b} \\ & = \frac{2(a+h)\pi}{2b\pi} = \frac{a+h}{b} \end{aligned} \quad (13)$$

For $h = H$ this results in ≤ 1.007 . Given the small numbers and the fact that an over-approximation of the result does not affect correctness, a simple implementation could simply consist of multiplying the radius by 1.005 to account for possible false negatives. It is probably advisable to account for this on the client's side to facilitate auditing. Otherwise a server might claim to not be malicious but return wrong results due to wrong approximations.

B. Database Tables and Schemas

When using a 2D spatial index as described in Section III-A, the tables `nodes` and `certificates` are used. The table `nodes` has the following columns:

- `bit_string` of type `bit` varying (66)
Stores the bit strings described in Sections II-A and II-C. The table is clustered by the value resulting in geographically close rows being stored somewhat close together on disk.
- `area` of type `geography`
Stores the 2D projection of the voxel corresponding to the row's `bit_string` in the database's internal format. There is a 2D spatial index on this column.
- `altitude_min` of type `smallint` | `null`
Stores the minimum altitude corresponding to the row's `bit_string`.
- `altitude_max` of type `smallint` | `null`
Stores the maximum altitude corresponding to the row's `bit_string`.
- `certificate_hashes` of type `bytea[]`
Stores the hashes of the certificates belonging to this node and is by default an empty array. The hashes are foreign keys to the `certificates` table.
- `neighbor_hash` of type `bytea` | `null`
Stores the hash of the node corresponding to the row's neighbor in the SMT (flipping the last bit in the row's bit string) if it is non-empty and otherwise is equal to `null`. This hash might be required for the SMT proof when this row is retrieved.
- `left_child_hash` of type `bytea` | `null`
Stores the hash of the node corresponding to the row's left child in the SMT (adding a 0 to the row's bit string) if it is non-empty and otherwise is equal to `null`. This hash might be required for the SMT proof when this row is retrieved.
- `right_child_hash` of type `bytea` | `null`
Stores the hash of the node corresponding to the row's right child in the SMT (adding a 1 to the row's bit string) if it is non-empty and otherwise is equal to `null`. This hash might be required for the SMT proof when this row is retrieved.

All columns other than `bit_string` and `certificate_hashes`, are redundant and could be computed based on other information inside the database but are stored explicitly to speed up queries. To ensure consistency, constraints and triggers are used wherever possible.

The `certificates` table only consists of two columns, the primary key `certificate_hash` of type `bytea` storing the SHA256 hash of the certificate and the column `certificate`, also of type `bytea`, storing the certificate itself.

IV. OPTIMIZATIONS

First of all it should be noted that the asymmetric design of the bit string with respect to the z dimension allows placing certificates higher in the tree and thus increase sparsity.

Under the assumption most of the world is rather flat with respect to altitude, certificates can often be placed at intermediate nodes covering the full height. This lowers the

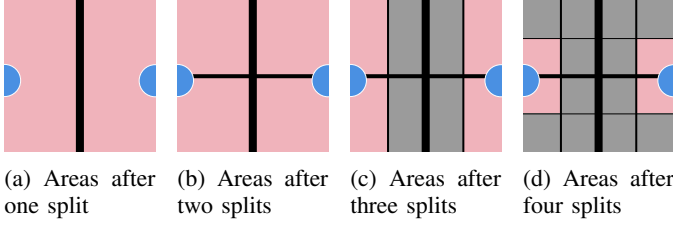


Fig. 5: The two-dimensional surface (longitude, latitude) after the first, second, third and fourth split. The blue circle shows a possible area to be claimed by a certificate or the desired query area of a client. The area colored in red is the minimum area that has to be retrieved using the given number of splits.

tree's height and thus reduces the proof sizes. For this to work, clients must not expect the certificates to be placed at a unique node in the tree. The placement does not matter as long as the certificate is stored with *all nodes at one level* whose geometric representation intersect the certificate's claimed volume. This is because a GeoPKI map server always returns the data along the path to the root together with the whole subtree. Hence when the certificate is placed in the subtree of the queried volume it will always be included but if it is placed higher in the tree it will only be included if the certificate is stored at one of the parent nodes.

Section II-E describes how the map server only serves an answer to a single bit string query. While such an API works, there is room for improvement if correlated queries are answered at the same time. For instance when the client derives the different bit strings to query a single volume as described in Section II-F, the queries share the path to the root from their lowest common ancestor. By replying with a single answer for such a set of queries, the data along this shared path has to be returned just once. Given the use of the Z-order curve (see Section II) to order the leaves, points spatially close should also be somewhat close in the tree and thus have a common ancestor on lower levels. Though in the worst case it is still possible that the only common ancestor is the root even if the two areas are spatially close. This happens when the volume covers both sides of the first split done in the binary tree. This scenario is depicted in Figure 3a. It seems impossible to avoid such a worst-case scenario entirely when using a tree as described in Section V.

Considering that earth is a sphere and that the coordinates wrap around, the situation becomes even more difficult since a similar situation happens at the points where the coordinate system wraps around. This scenario is depicted in Figure 5. This wrap-around can be interpreted as being part of the first split since the problem occurs exactly on the opposite side of the first split when considering the x -axis as wrapping around. To reduce it's impact it would be possible to shift the longitude coordinate system west by 30° . This way this circle around earth passes through the Atlantic Ocean and Pacific Ocean rather than central Europe.

An additional optimization would be to set B_y to 25 rather

than 26 bits. This slightly complicates the notations, definitions and formats but is otherwise straight-forward to implement.

V. ALTERNATIVE DESIGNS

An alternative design could use a (hybrid-)quadtree or (hybrid-)octree where most intermediate nodes have four or eight children, respectively. "Hybrid" and "most" because $B_x = B_y > B_z$ which means the volume cannot be subdivided fully using a quad- or octree. Hence when using an octree with eight children, the z dimension is already exhausted after B_z splits and will from there continue to only have four children (i.e. a quadtree). When using a quadtree, the first $B_{x,y}$ splits would solely concern the x and y dimensions and after that the children will be binary trees of height B_z only splitting the z dimension. This will decrease the tree's height but increase the proof size due to the larger number of neighboring hashes that have to be provided on each level. For the hybrid-quadtree, the tree height would be $\log_4(2^{2 \cdot B_{x,y}}) = \log_4(4^{B_{x,y}}) = B_{x,y} = 26$ plus the $\log_2(2^{B_z}) = B_z = 15$ for the binary subtrees. For the hybrid-octree the tree height would be $\log_8(2^{B_z}) = \log_8(8^{B_z/3}) = B_z/3 = 5$ plus the $\log_4(2^{B_{x,y}}) = \log_4(4^{B_{x,y}}) = B_{x,y} = 26$ for the quad-subtrees. The proof size of a leaf would thus increase to $(26 \cdot (4 - 1) + (15 - 1) \cdot (2 - 1)) \cdot 32 \text{ B} = 2'944 \text{ B} \approx 3 \text{ kB}$ and $(5 \cdot (8 - 1) + (26 - 1) \cdot (4 - 1)) \cdot 32 \text{ B} = 3'520 \text{ B} \approx 3.5 \text{ kB}$, respectively. The ones are subtracted from the tree height since the on the last level no complementary hash has to be returned. The ones subtracted from the number of neighbors is due to the fact that the hash resulting from the leaf itself does not have to be provided as it can be computed by the client.

Unfortunately the worst case scenario remains: Two spatially adjacent volumes intersecting the lines of the first split only share the root node as a common ancestor. When using a tree this will always be the case due to nodes having a single parent. Consider single dimensional discrete consecutive numbers and a binary tree. Each number (except the start and end) has two numerically neighbors but can only have one neighbor in the tree. This fact also holds for group of leaves and thus there will always be two numerically close neighbors just sharing the root node as their ancestor.

To maintain consistency with the already introduced binary representation of volumes, one can consider the quad-tree part (i.e. the first $B_{x,y}$ levels) to add two bits to the prefix on each level rather than just one. The last B_z levels would then each just add a single bit as before.

Since it is not yet clear if and to what extent the usage a more complex tree structure could be beneficial and hence the more simple binary tree is preferred for the moment.

VI. CAVEATS

Since longitude (x) and latitude (y) coordinates that are split in half numerically represent polar coordinates on a sphere, numerically equidistant points, areas or volumes are not equidistant on earth. Near the poles the longitudinal circumference is smaller but represented using the same numerically range resulting in higher precision. Hence the discretization range

was chosen based on the semi-major axis at the equator where the achieved precision is smallest.

VII. CERTIFICATE FORMAT

The claimed volume has to be specified in the X.509 certificate. The used format should be flexible enough to support a variety of use cases such as a set of disjoint volumes, rectangular but also circular volumes. Open Street Maps uses polygons and RFC 7946 specifies the “geospatial data interchange format” GeoJSON [8] that also makes use of polygons. One option would be to define additional types allowing for three dimensional polygons, another to use the “properties” field defined in RFC 7946. When going with the latter approach, each certificate could define a `GeometryCollection of Polygons` or `MultiPolygons`, each with an `altitude_from` and `altitude_to` property indicating the inclusive minimum and maximum altitude, respectively. Note that this representation only allows for extruded two dimensional polygons as primitives but their composition allows more complex shapes. Since the GeoJSON uses a text-encoding, serializing the data using e.g. protocol buffers as done by Mapbox [9] will result in a smaller encoding and fast read and write operations.

REFERENCES

- [1] NGA geomatics - WGS 84. [Online]. Available: <https://earth-info.nga.mil/?dir=wgs84&action=wgs84>
- [2] N. O. a. A. A. US Department of Commerce. What is the highest point on earth as measured from earth’s center? [Online]. Available: <https://oceanservice.noaa.gov/facts/highestpoint.html>
- [3] S. K. Dhaka and V. Kumar, “Chapter 1 - composition and thermal structure of the earth’s atmosphere,” in *Atmospheric Remote Sensing*, ser. Earth Observation, A. Kumar Singh and S. Tiwari, Eds. Elsevier, pp. 1–18. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/B9780323992626000237>
- [4] J. V. Gardner, A. A. Armstrong, B. R. Calder, and J. Beaudoin, “So, how deep is the mariana trench?” vol. 37, no. 1, pp. 1–13, publisher: Taylor & Francis _eprint: <https://doi.org/10.1080/01490419.2013.837849>. [Online]. Available: <https://doi.org/10.1080/01490419.2013.837849>
- [5] P. Kabat, W. van Vierssen, J. Veraart, P. Vellinga, and J. Aerts, “Climate proofing the netherlands,” vol. 438, no. 7066, pp. 283–284, number: 7066 Publisher: Nature Publishing Group. [Online]. Available: <https://www.nature.com/articles/438283a>
- [6] L. Chuat, C. Krähenbühl, P. Mittal, and A. Perrig, “F-PKI: Enabling innovation and trust flexibility in the HTTPS public-key infrastructure,” in *Proceedings 2022 Network and Distributed System Security Symposium*. [Online]. Available: <http://arxiv.org/abs/2108.08581>
- [7] G. M. Morton, “A computer oriented geodetic data base; and a new technique in file sequencing.”
- [8] H. Butler, M. Daly, A. Doyle, S. Gillies, T. Schaub, and S. Hagen, “The GeoJSON format,” num Pages: 28. [Online]. Available: <https://datatracker.ietf.org/doc/rfc7946>
- [9] Mapbox, “Geobuf,” original-date: 2014-07-14T20:27:58Z. [Online]. Available: <https://github.com/mapbox/geobuf>